Instacart for customers and their AI assistants

A customer describes what they need. Their own AI assistant handles the shopping.

Case premise: agents may act at scale within 12 months, often without a person reviewing every step.

The customer's expectation

"Keep my weekly groceries stocked, within my preferences and budget."

Weekly orders, substitutions and recurring lists

Instacart needs to support both people and external assistants acting on their behalf.

The challenge created by delegation

Consistent meaning

The same product, quantity and preference must mean the same thing across different assistants.

Clear permission

Routine shopping must stay within customer-approved choices and spending. Unclear requests need confirmation.

Changing conditions

Stock and prices can change. A replacement or adjustment must still fit what the customer authorised.

The opportunity: less shopping effort, with orders that match the customer's intent and limits.

Metrics and trade-offs behind my recommendations

I used the case constraints to choose a direction and defined what evidence could change it.

These are proposed measures and decision criteria. The case provides no measured results.

What I assess	Metrics and evidence	How it informs the choice
Customer value	Corrections per repeat order; voluntary repeat-use rate.	Test whether repeat shopping needs fewer corrections.
Journey effort	Ready-basket abandonment; checkout vs replacement minutes.	Prioritise the greater reduction in effort or lost orders.
Order correctness	Intended-basket match across agents; failure rate and severity.	Find the cause, fix it and retest earlier cases.
Purchase risk	Unintended / all accepted autonomous orders; money lost.	Adjust caps or pause autonomy when permission fails.
Value and cost	Funding relevance; build effort, expected rework and support cost.	Accept demo rework when funding value justifies the cost.
Readiness	Unresolved dependencies; evidence that pause and recovery work.	Limit commitments to what our evidence supports.

How I judged each choice

Assumptions stated

Q2: funding matters. Q3: preferences first, customer completes checkout in Instacart. Q5: provisional caps.

Trade-offs accepted

Demo effort can be discarded. Preferences first retains customer checkout. Initial caps add confirmation.

Conditions for changing course

Switch when another option offers more value or fewer failures. Pause autonomy for permission breaches.

Reasoning flow Unpack need, goal and constraints → Diagnose causes → Choose metrics → Compare options → Decide trade-off → Review

Why groceries? A rich learning environment for shopping agents

Product variety tests capability. Recurring purchases create opportunities to improve.

One category tests many shopping skills

Varied products	Labels, images and similar variants test precise product understanding.
Mixed quantities	Count, weight and pack size test exact amounts, e.g. 2 x 500 g.
Prices and offers	Unit prices, discounts and sale history test better-value choices.
Changing stock	An unavailable usual item tests adaptation within the user's rules.
Linked basket items	Shared meal ingredients test coordination across multiple items.
Timing constraints	Different shelf lives and repeat needs test purchase timing.

Measure improvement across repeat orders

Accuracy: product, quantity and price/promotion correctness.

Progress: fewer corrections on comparable orders; voluntary repeat use.

Frequent purchases let us test improvement sooner

Weekly groceries give the same household an earlier chance to evaluate improvements than an occasional purchase.

Confirmed corrections can improve later baskets. Test whether effort falls and voluntary repeat usage grows.

WHY? Earlier evidence of useful, reliable agent decisions.

Compared with	Why start with groceries?
Laptop / leisure trip	More frequent comparable cycles for the household use case.
Clothing purchase	Units and product attributes are easier to verify than subjective fit.
Non-food consumables	Groceries add meal dependencies and perishability; consumables may be simpler.

Groceries combine diverse decisions with frequent feedback, making them a strong starting point for shopping agents.

I would build a focused prototype for the six-week board demo.

WHY?

Assumption: the six-week demo materially influences funding for the next stage.

Temporary code can be worthwhile if it helps secure investment. The demo must add enough value to justify its cost.

Option	Business value	Cost or risk
Build the requested feature	May impress the board and secure support.	Expected rebuild within three months. Extra scope needs a clear funding benefit.
Delay the demo	More engineering effort goes toward the durable product.	Could lose funding or momentum. Technical progress alone may not offset that loss.
My choice: focused prototype	Show the customer value needed for the funding decision.	Accept agreed temporary effort. Explain the limitations and remaining production investment.

Evidence behind my choice

Funding relevance

Would a focused demo answer the same investment question as the requested feature?

Cost of the choice

Engineering weeks spent, likely rebuild effort and production work displaced.

Evidence of future failure

Failure rate and severity with realistic agent requests. Small fixes or a rebuild?

Board expectations

What does the demo prove, and what remains unproven before launch?

The trade-off I accept

Some temporary work and slower production progress, in exchange for preserving the funding opportunity.

Meet the board deadline. Push back on costly scope and unsupported launch commitments.

What each API does, and what happens without it

Substitution-preference API

Lets the agent read the customer's replacement choices, such as a saved backup or a refund.

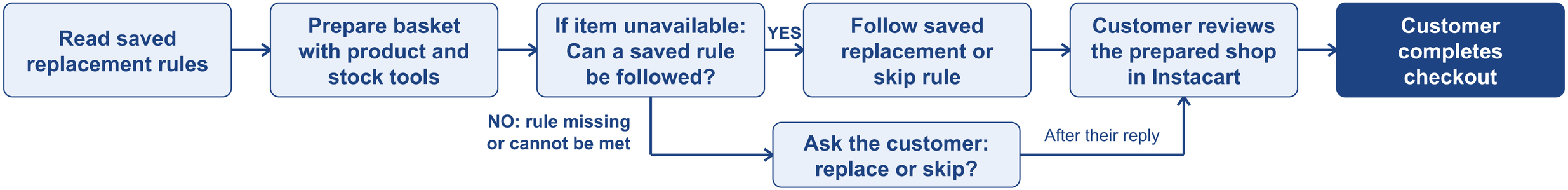
It supplies rules. Product tools find the alternatives.

Checkout API

Lets the agent turn a ready basket into an order, with the customer's permission to buy.

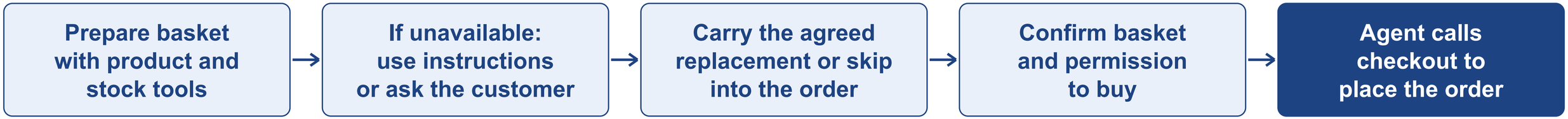
It places the order. It does not validate every choice.

A. Preferences first: the agent cannot use checkout



Dependency: a working handoff to Instacart. The customer still completes the purchase.

B. Checkout first: the agent cannot read saved preferences



What remains: replacement questions when instructions are missing. Agreed choices need a separate way to reach fulfilment.

Stock can change during picking too. Replacement rules still matter after checkout.

I would make substitution preferences agent-readable first

WHY?

Saved substitution preferences can reduce repeated replacement questions.

Trade-off: the agent prepares the basket. The customer places the order in Instacart.
Assumption: existing checkout works and saved replacement choices reach the shopper.

The comparison behind my choice

Decision factor	Checkout first	Preferences first
Role in the journey	Completes purchase once a basket is ready. A customer can do this for now.	Guides replacement choices during basket preparation and later during picking.
What work can it remove?	Checkout actions on each purchase. Review and buying permission still matter.	Repeated replacement choices across items and weekly shops.
Where is the limit?	Placing an order does not settle later replacement decisions.	Reading rules does not fix all basket errors or complete the purchase.

Why I would improve replacement decisions first

Resolve known replacement choices before buying. With human checkout available, I prioritise those decisions. Trade-off: slower completion and no autonomous ordering yet.

Metrics that could change the choice

Completion lost at handoff

Share of ready baskets abandoned or rebuilt during customer checkout. More loss favours checkout.

Customer effort per shopping attempt

Compare checkout time with replacement frequency and question time. Prioritise the larger saving.

Orders that match user intent

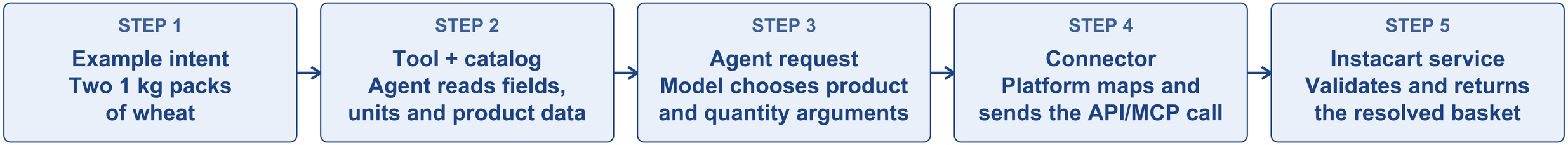
Share fulfilled within approved item and spending rules. Less effort must not mean worse orders.

Preferences first. Switch if checkout blocks more purchases or saves more customer effort.

Agent workflow: where can the meaning of "2" change?

The case: quantity: 2 becomes "2 units" on Platform A and "two 1 kg packs" on Platform B.

Leading hypothesis: an unclear shared definition. Quantity is one example; the same fault can affect other fields.



Test the same purchase on both platforms → find the cause → fix → retest, including cases that passed before.

Where to inspect	Evidence that locates the fault	What must change
Step 2: Definition / catalog	The count basis, pack size or supported definitions differ.	Instacart fixes the shared meaning and catalog data; align API, MCP and versions.
Step 3: Agent arguments	Definitions are clear; the agent chooses the wrong product or amount.	Correct tool guidance and product selection; validate the generated arguments.
Step 4: Connector mapping	Arguments are right before mapping, wrong when the API receives them.	Fix the adapter conversion or version mismatch; preserve product and quantity meaning.
Step 5: Server result	The correct request and fixed context produce the wrong basket.	Fix service quantity handling; check the resolved product, count and physical amount.

Define exactly what quantity counts, enforce it in the API, and verify both agents produce the intended basket.

Start with provisional caps. Personalise as evidence grows.

WHY?

Use shopping history for a starting cap, then learn from agent outcomes.

First 30 days: 0-5 orders without fresh confirmation. No agent-purchase history means no proven safe range.

The initial monthly threshold

Usual orders / month	Starting cap
No usable history or no opt-in	0: confirm each purchase
1-2	1
3-4	1-2
5-9	2-4
10+	5

H: median of the last three 30-day human order counts.

Starting rule: half H, rounded down. Minimum 1 for eligible users, maximum 5 or the user's lower limit.

Half limits exposure. Five fits five weekly dates in 30 days. Both are provisional choices.

How models improve the recommendation

Shopping need

Estimate likely frequency from recent intended purchases and recurring preferences. Human and agent orders form one demand history.

Unintended-order risk: a later model

Candidate: logistic regression to estimate the chance of an unwanted order. Learn from explicit feedback, timing, basket / price changes and corrections.

Output: estimated need, order risk and uncertainty. A policy turns these into a monthly cap recommendation within approved limits.

Train on enough relevant agent-order feedback and validate added order types before changing the policy.

Start simple. Adopt a model only when relevant data shows better outcomes.

Models inform the cap. Customer limits govern every order.

Limits the model cannot override

Permission and duplicate prevention

Opt-in, approved shopping occasions and one order per occasion. The account cap covers every agent.

Price and spending

User-approved limits per order and per rolling 30 days, including fees and allowed substitutions.

Products, quantities and replacements

Stay within approved preferences. An exception or uncertain intent requires confirmation, even below the cap.

Price still changes the usable allowance

Example: count cap 3; approved limits $100 / order and $180 / rolling 30 days.

3 x $60 = $180

2 x $90 = $180

A third $90 order needs approval despite a free count slot. A model score cannot increase the budget.

How evidence changes the monthly cap

Evidence at review	Decision
Repeated unmet demand, validated low risk for added orders and acceptable total spending exposure.	Propose a higher cap with fresh consent.
Too little data, uncertain feedback or no repeated extra demand.	Keep the current cap.
The user wants less or shopping needs fall.	Lower the agreed cap.
An unintended purchase or a permission / spending breach.	Set cap to 0. Confirm purchases during review.

Review at 30 days. More time may be needed before a model is reliable.

Adoption test: improve on the starting policy

Assess unintended-order rate, money lost and confirmations per intended order. More orders alone are not success.

No complaint does not mean the purchase was intended.

Personalise the recommendation as evidence grows. Keep consent and spending limits firm.

Our framework for identifying and managing project risk

MY REPLY “I understand the concern. Our framework focuses on identifying risks early and recommending how to address them. We will use relevant competitor findings to challenge our approach, correct any gaps and verify the fixes. You will see the evidence behind our launch recommendation, with clear ownership of any remaining risks.”

Identify risks early

Assess our project data. Use any verified competitor findings to check for shared causes and recommend a response.

If we find a gap: own the correction and verify the fix against agreed criteria for order accuracy, permissions and dependencies.

Weigh the value and risk

Compare demonstrated customer benefit with build, operating, support and failure costs.

Decision: limit release to validated scope. Narrow or hold if value is unproven or critical safeguards are not demonstrated.

Contain and recover

Limit initial exposure. Prove that pause and recovery work before expanding.

If an issue occurs: pause the affected agent capability, keep unaffected shopping services running, and provide a tested alternative to help customers complete or correct their orders.

Our accountability: delivery quality, validation, early risk disclosure, a clear recommendation and correcting our errors.

Instacart’s decision: approve the business launch with remaining risks explicit. Sign-off does not remove our accountability.

Our commitment: identify risks early, recommend a response and take ownership of corrections.

Thank you

Instacart Agent Readiness Case Study